

Tier 1 + Tier 2 Study Companion

Competitive programming up to CF ~1700 — triggers, implementations, modern C++, and intuition

This document is the **study material** for Tiers 1 and 2 of the roadmap (Phases 0–2). Tier 1 sections are deliberately technical and compact — recognition triggers, idiomatic modern C++, and the pitfalls that actually cost contests. Tier 2 sections add shaded **Intuition** blocks: the mental model, why the method is correct, and why the complexity is what it is, so the technique is understood rather than memorized.

How to use it (this matters more than the content). Per the roadmap method: read *one* section (20–40 min), close the document, implement from memory, then solve 3–5 focused problems and 2–3 mixed ones before moving on. This document is the read-and-reference step. It cannot replace implementing; nothing can.

Contents

Part 0 — Setup: 0.1 Base template · 0.2 Modern C++ (17/20/23) in contests · 0.3 Stress-testing rig · 0.4 Complexity budgeting

Part 1 — Tier 1: 1.1 STL containers · 1.2 STL algorithms · 1.3 Bits · 1.4 Sorting & comparators · 1.5 Binary search · 1.6 Two pointers · 1.7 Sliding window · 1.8 Prefix sums · 1.9 Difference arrays · 1.10 Brute force · 1.11 Recursion · 1.12 Backtracking · 1.13 Greedy basics · 1.14 Divide & conquer · 1.15 Modular arithmetic · 1.16 GCD/LCM · 1.17 Binary exponentiation · 1.18 Sieve (+SPF) · 1.19 Trial division

Part 2 — Tier 2: 2.1 Graph representation · 2.2 BFS · 2.3 DFS · 2.4 Components & flood fill · 2.5 Bipartite check · 2.6 Cycle detection · 2.7 Topological sort · 2.8 DSU · 2.9 Monotonic stack · 2.10 Monotonic deque · 2.11 Thinking in DP · 2.12 Linear DP · 2.13 Knapsacks · 2.14 Coin change · 2.15 LIS · 2.16 LCS · 2.17 Grid DP · 2.18 2D prefix sums/diff · 2.19 Prefix-sum DP optimization · 2.20 Extended Euclid · 2.21 Modular inverse · 2.22 Factorials & nCr · 2.23 Stars and bars · 2.24 Catalan numbers · 2.25 Derangements · 2.26 Inclusion–exclusion · 2.27 Overflow · 2.28 Divisors · 2.29 Strings: matching & palindromes · 2.30 Polynomial hashing · 2.31 Coordinate compression · 2.32 Indexing & precision · 2.33 Structs & comparators · 2.34 Interactive problems · Exit checklist

PART 0 — SETUP

0.1 The base template

Submit with the newest 64-bit GNU G++ option the judge offers (Codeforces has a C++23 option). One template, memorized:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

void solve() {
    // per-testcase code
}

int main() {
    ios_base::sync_with_stdio(false); // unties C++ streams from C stdio -> much faster
    cin.tie(nullptr); // no flush of cout before every cin
    int t = 1;
    cin >> t; // delete this line for single-test problems
    while (t--) solve();
}
```

- Print `'\n'`, never `endl` in loops — `endl` flushes the buffer every time and can alone cause TLE. (Exception: interactive problems, §2.34, where flushing is the point.)
- `bits/stdc++.h` is GCC-only and includes everything; standard in CP, not in production code.

- Reset global state between testcases — a leftover visited-array from the previous case is a classic wrong-answer. Prefer declaring containers inside `solve()` sized exactly for the case.
- Local compile for debugging: `g++ -std=c++23 -O2 -g -fsanitize=address,undefined -D_GLIBCXX_DEBUG a.cpp` — sanitizers catch out-of-bounds and overflow at the cost of speed; compile without them for timing runs.

0.2 Modern C++ (17/20/23) that actually matters in contests

Use the subset that removes boilerplate and bugs; ignore the rest. Everything below compiles on current judges (GCC 13/14); the two items marked GCC 14 need the newer compiler — each has a portable fallback shown later.

```
// structured bindings (C++17): unpack pairs/tuples/structs
auto [x, y] = pts[i];
for (auto& [key, val] : mp) val++;

// CTAD (C++17): template args deduced
vector dp(n + 1, vector<ll>(m + 1, 0));          // vector<vector<ll>>
pair p{1, 2.5};

// ranges (C++20): whole-container algorithms + projections
ranges::sort(v);
ranges::sort(v, greater{});                      // descending
ranges::sort(pts, {}, &Point::x);                // sort by one member ("projection")
auto it = ranges::lower_bound(v, x);
int mx = ranges::max(v);

// <numeric>, <bit> (C++17/20)
ll g = gcd(a, b), l = a / gcd(a, b) * b;          // std::gcd / std::lcm
int mid = midpoint(lo, hi);                       // overflow-safe (lo+hi)/2
int bits = popcount(x);                          // x must be UNSIGNED, see 1.3

// containers (C++20)
if (mp.contains(key)) ...                        // no accidental insertion
erase_if(v, [](int x) { return x % 2 == 0; });    // remove + erase in one call

// C++23 niceties (newer judges)
if (s.contains("abc")) ...                      // string::contains
for (auto [i, x] : views::enumerate(v)) ...     // index + value
```

- **What to skip:** heavy views pipelines, `std::format/std::print` (fast `cout` is enough), smart pointers, exceptions. Contest code optimizes for writing speed and debuggability.
- When unsure whether a feature exists on the judge, the classic iterator form (`sort(v.begin(), v.end())`) always works — know both spellings.

0.3 The stress-testing rig (build once, use forever)

Highest-value tool in this entire document. When your solution fails and you can't see why: write a slow-but-obviously-correct brute force, generate small random inputs, and diff the outputs until a counterexample appears. Failing tests shrink from "invisible" to "8 numbers you can trace by hand".

gen.cpp — random input from a seed (seed as argument makes failures reproducible):

```
#include <bits/stdc++.h>
using namespace std;
int main(int argc, char** argv) {
    mt19937 rng(atoi(argv[1]));
    auto rnd = [&](int lo, int hi) { return uniform_int_distribution<int>(lo, hi)(rng); };
    int n = rnd(1, 8);                      // SMALL sizes: bugs live in small cases
    cout << n << "\n";
    for (int i = 0; i < n; i++) cout << rnd(1, 20) << " \n"[i == n - 1];
}
```

brute.cpp — the dumbest correct solution you can write (try all subsets, all pairs, simulate). Correctness over speed.

stress.sh:

```
#!/bin/bash
g++ -O2 -o sol sol.cpp && g++ -O2 -o brute brute.cpp && g++ -O2 -o gen gen.cpp || exit
for ((i = 1; ; i++)); do
    ./gen $i > in.txt
    ./sol < in.txt > out_sol.txt
    ./brute < in.txt > out_brute.txt
    if ! cmp -s out_sol.txt out_brute.txt; then
        echo "FAILED on seed $i:"; cat in.txt
        echo "-- sol:"; cat out_sol.txt; echo "-- brute:"; cat out_brute.txt
        break
    fi
    echo "ok $i"
done
```

- Use mt19937 seeded from the argument — never `rand()` (weak, and unseeded runs repeat).
- Adapt gen per problem: random trees (each node picks a parent among earlier nodes), random strings, permutations via `shuffle`. Keep constraints tiny first; only enlarge once small cases pass.
- If outputs can differ but both be valid (multiple answers), replace `cmp` with a third program — a *checker* that reads input + output and verifies validity.

0.4 Complexity budgeting

Before designing anything, read the constraints and compute the target complexity. Rule of thumb: roughly 10^8 – 10^9 simple operations per second on Codeforces; budget $\sim 10^8$ per second to be safe (cache misses, divisions, and heavy constants eat the rest).

n up to ~ 10 – 11	$O(n!)$	permutations
n up to ~ 20 – 24	$O(2^n * n)$	subsets / bitmask
n up to ~ 100	$O(n^4)$	
n up to ~ 500	$O(n^3)$	
n up to $\sim 5,000$ – $10,000$	$O(n^2)$	
n up to $\sim 2 \cdot 10^5$ – 10^6	$O(n \log n)$ or $O(n)$	
n up to $\sim 10^8$	$O(n)$ simple loop	
n $\sim 10^9$ – 10^{18}	$O(\sqrt{n})$, $O(\log n)$, $O(1)$	$\rightarrow$ math / binary search

Working backwards from this table is the first solving step: if $n \leq 5000$, an $O(n^2)$ DP is *allowed* — that is a hint, not a coincidence. Setters choose constraints to admit the intended solution and kill the naive one.

PART 1 — TIER 1 (CF ~ 800 – 1200): the technical core

Goal per the roadmap: every item codes from memory in under 3 minutes. Minimal theory, maximal precision.

1.1 STL containers — operations, complexities, traps

`vector` — the default container. `push_back` amortized $O(1)$, random access $O(1)$, insert/erase in the middle $O(n)$. `v.assign(n, x)` to resize-and-fill; `v.back()`, `v.pop_back()`.

- Iterators/references are invalidated by reallocation — never hold a reference across a `push_back`. Growing a vector you are iterating over is undefined behavior.
- `vector<bool>` is bit-packed: tiny memory, but elements aren't real `bool` — fine for visited arrays, avoid taking references into it. `basic_string<char>`-style alternative: `vector<char>`.

`string` — a vector of chars with extras: `substr(pos, len)` $O(len)$, `s + t` allocates ($O(\text{total})$), `s += t` amortized. Compare lexicographically with `<`. Building a string by repeated `s = s + ch` is $O(n^2)$; use `s += ch`.

`stack` / `queue` / `deque` — `stack`: push/pop/top; `queue`: push/pop/front; `deque` does $O(1)$ push/pop at *both* ends plus random access. In practice a plain vector with `push_back/pop_back` is the fastest stack.

unordered_map / unordered_set — average O(1) insert/find/erase, but worst case O(n): on Codeforces, adversarial tests *do* break the default hash for integer keys. Below ~1700 the safe default is map or sorting; if you need hashing speed, plug in a randomized hash:

```
struct chash {
    // randomized splitmix64 -- unhackable
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t R = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + R);
    }
};
unordered_map<ll, int, chash> mp;
```

map / set / multiset — balanced BSTs: O(log n) everything, iteration in sorted order.

set::lower_bound(x) / upper_bound give the first element $\geq / > x$ — use the *member* versions (the free-function ones are O(n) on sets). prev(it) / next(it) step neighbors.

- mp[key] **inserts** a default value if absent — perfect for counting (cnt[x]++), dangerous for querying. Query with contains / find / count.
- ms.erase(value) on a multiset erases **all** copies. One copy: ms.erase(ms.find(value)).

priority_queue — a binary heap: O(log n) push/pop, O(1) top. **Max**-heap by default.

```
priority_queue<int> mx; // max on top
priority_queue<int, vector<int>, greater<>> mn; // min on top
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq; // by .first then .second
```

pair / tuple — compare lexicographically (first, then second, ...), which makes sort on pairs "sort by key, tie-break by value" for free. Unpack with structured bindings.

1.2 STL algorithms — the working catalog

```
sort(v.begin(), v.end()); // ranges::sort(v);
reverse(v.begin(), v.end()); // ranges::reverse(v);
v.erase(unique(v.begin(), v.end()), v.end()); // dedupe -- MUST be sorted first
auto it = lower_bound(v.begin(), v.end(), x); // first element >= x (sorted!)
auto jt = upper_bound(v.begin(), v.end(), x); // first element > x
int cnt_x = jt - it; // occurrences of x
int mx = *max_element(v.begin(), v.end()); // ranges::max(v)
ll sum = accumulate(v.begin(), v.end(), 0LL); // 0LL !! -- '0' sums in int and overflows
fill(v.begin(), v.end(), 0);
iota(v.begin(), v.end(), 0); // 0, 1, 2, ...
int c = count(v.begin(), v.end(), x);
auto f = find(v.begin(), v.end(), x); // O(n); on set/map use the member find
nth_element(v.begin(), v.begin() + k, v.end()); // k-th smallest in O(n), rest unordered
ranges::sort(v); v.erase(ranges::unique(v).begin(), v.end()); // C++20 ranges dedupe
```

- The single most common Tier-1 bug in this list: accumulate(..., 0) summing 10^5 values of 10^9 each in an int. Write 0LL reflexively.
- lower_bound/upper_bound on unsorted data silently return garbage — they binary-search and *assume* order.

1.3 Bits: <bit> and manipulation

Modern best practice is the C++20 <bit> header (type-safe, works for 64-bit without remembering ll-suffixed builtins). All of these require **unsigned** arguments — cast.

```
unsigned long long x = ...;
popcount(x) // number of set bits (__builtin_popcountll)
countl_zero(x) // index of lowest set bit (__builtin_ctzll) x != 0
countl_zero(x) // leading zero count (__builtin_clzll) x != 0
```

```

bit_width(x)      // 1 + floor(log2 x); bit_width(0) == 0
bit_floor(x)      // largest power of two <= x
bit_ceil(x)       // smallest power of two >= x
has_single_bit(x) // is x a power of two

// manipulation cookbook (works on signed too):
x >> i & 1        // read bit i
x |= 1LL << i     // set bit i
x &= ~(1LL << i)  // clear bit i
x ^= 1LL << i     // toggle bit i
x & -x            // value of lowest set bit
x & (x - 1)       // x with lowest set bit removed
(x & (x - 1)) == 0 && x // power of two test

```

- **The classic overflow:** $1 \ll k$ is an int — undefined for $k \geq 31$. Any 64-bit shift must be $1LL \ll k$. This bug passes samples and fails test 9.

1.4 Sorting and comparators

Triggers. "Order of processing doesn't matter" → sort and exploit structure; "pair each smallest with..."; any greedy that says "by deadline / by size / by ratio". Sorting first is the most common preprocessing step in CP.

```

ranges::sort(v); // ascending
ranges::sort(v, greater{}); // descending
ranges::sort(pts, {}, &Point::x); // by member (projection)
ranges::sort(a, [](auto& p, auto& q) { // custom comparator
    return p.second != q.second ? p.second < q.second : p.first < q.first;
});
stable_sort(v.begin(), v.end(), cmp); // keeps order of equals

```

- **Comparator contract (strict weak ordering):** `cmp(a, a)` must be false. Writing `return a.x <= b.x;` is undefined behavior — it crashes or corrupts the sort on real tests. Always `strict <`.
- Comparing by a ratio a/b : avoid doubles; compare cross-products $a_1*b_2 < a_2*b_1$ in `ll`.
- Often you don't sort the data — you sort the **indices**: `iota(idx); ranges::sort(idx, {}, [&](int i){ return a[i]; });` keeps original positions recoverable.

1.5 Binary search

Triggers. Sorted data + "find / count"; "largest value such that..." / "smallest value such that..."; a yes/no question that is monotone in its parameter. One mental model covers every variant: **the predicate framework**. Arrange the search space so that some predicate is false false ... false true true ... true; binary search finds the boundary.

```

// first index in [lo, hi) where pred(i) is true; returns hi if none
// requires: pred is monotone (false...false true...true)
int lo = 0, hi = n;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2; // or: midpoint(lo, hi) -- overflow-safe
    if (pred(mid)) hi = mid;      // boundary is at mid or left of it
    else lo = mid + 1;           // boundary is strictly right of mid
}
// here lo == hi == first true position

```

- Invariant view: `pred(hi)` is always true-or-past-the-end, `pred(lo-1)` always false. Every step shrinks `[lo, hi)` — no infinite loops possible in this form, which is why you should use exactly this form every time.
- "Last false" is just `first_true - 1`. Don't write a second template; reuse one.

```

// library forms on a sorted vector:
int i = lower_bound(v.begin(), v.end(), x) - v.begin(); // first >= x
int j = upper_bound(v.begin(), v.end(), x) - v.begin(); // first > x
// elements < x: i      elements <= x: j      elements == x: j - i

// real-valued search: fixed iteration count beats epsilon fiddling

```

```
double lo = 0, hi = 1e18;
for (int it = 0; it < 100; it++) {
    double mid = (lo + hi) / 2;
    if (pred(mid)) hi = mid; else lo = mid;
}
```

- Pitfalls: searching unsorted data; mixing up $\geq$ vs $>$ (know lower_bound/upper_bound cold); $(lo + hi) / 2$ overflowing when bounds are near 10^{18} — write $lo + (hi - lo) / 2$.

1.6 Two pointers

Triggers. Sorted array + pair condition ("two elements summing to X"); merging two sorted sequences; "for each left end, the furthest valid right end" when that right end only moves forward. The pattern: two indices that each move only one direction $\rightarrow$ total work $O(n)$, even though it looks like a nested loop.

```
// pair with sum == target in a sorted array
sort(a.begin(), a.end());
int i = 0, j = n - 1;
while (i < j) {
    ll s = (ll)a[i] + a[j];
    if (s == target) { /* found a[i], a[j] */ break; }
    if (s < target) i++; // sum too small: only increasing the small side can help
    else j--; // sum too big: only decreasing the big side can help
}

// merge two sorted arrays
int i = 0, j = 0;
while (i < n || j < m)
    if (j == m || (i < n && a[i] <= b[j])) out.push_back(a[i++]);
    else out.push_back(b[j++]);
```

- Correctness of the pair scan in one line: when $a[i] + a[j] < \text{target}$, no pair (i, j') with $j' \leq j$ can work either, so index i is fully resolved — advance it. Each step permanently discards one index.
- Same-direction two pointers is the skeleton of the sliding window (next section) — learn them as one idea.

1.7 Sliding window

Triggers. "Longest/shortest/count of **contiguous** subarrays with property P", where P is **monotone**: if a window is bad, every extension of it is bad ($\text{sum} \leq K$ with positive numbers, at most K distinct values, ...). Then for each right end the valid left ends form a range, and the left boundary only ever moves right $\rightarrow O(n)$.

```
// longest subarray with sum <= K (all a[i] > 0 !)
ll sum = 0; int best = 0;
for (int l = 0, r = 0; r < n; r++) {
    sum += a[r]; // extend right
    while (sum > K) sum -= a[l++]; // shrink left until valid again
    best = max(best, r - l + 1);
}

// longest substring with at most K distinct characters
array<int, 26> cnt{}; int distinct = 0, best = 0;
for (int l = 0, r = 0; r < n; r++) {
    if (cnt[s[r] - 'a']++ == 0) distinct++;
    while (distinct > K)
        if (--cnt[s[l++] - 'a'] == 0) distinct--;
    best = max(best, r - l + 1);
}
```

- Fixed-size window of length k : add $a[r]$, and when $r \geq k$ remove $a[r-k]$.
- **When it breaks:** negative numbers kill the monotonicity of " $\text{sum} \leq K$ " (shrinking can make the sum bigger). Then the window argument is invalid — reach for prefix sums + other tools instead.

Checking monotonicity before writing the window is the actual skill.

- Counting variant: if a window $[l, r]$ is the smallest valid one for this r , then "number of valid subarrays ending at r " is often $l + 1$ or $r - l + 1$ depending on direction of the property — derive it, don't guess.

1.8 Prefix sums (1D)

Triggers. A *static* array + many range-sum questions; "sum of subarray" inside a bigger algorithm; counting subarrays with a given sum (pair it with a hash map of prefix values).

```
vector<ll> p(n + 1, 0);
for (int i = 0; i < n; i++) p[i + 1] = p[i] + a[i];
// sum of a[l..r-1] (half-open [l, r)): p[r] - p[l]
// sum of a[l..r] (inclusive): p[r + 1] - p[l]
```

- Adopt the half-open convention with $p[0] = 0$ — it removes every off-by-one and special case for $l = 0$.
- The same trick works for any invertible operation: prefix XOR (XOR of range = $p[r] \oplus p[l]$), prefix products modulo a prime (range product = $p[r] * \text{inverse}(p[l])$, §2.21). It does NOT work for min/max — not invertible.
- Classic combo: "count subarrays with sum exactly S " → for each r , count earlier prefix values equal to $p[r] - S$ with a hash map. Memorize this pairing.

1.9 Difference arrays (1D)

Triggers. Many **range updates** ("add x to $a[l..r]$ ") and only at the end do you read the array. Mirror image of prefix sums: prefix sums turn range-reads into two point-reads; difference arrays turn range-writes into two point-writes.

```
vector<ll> d(n + 1, 0);
// for each update "add x on [l, r]" (inclusive, 0-indexed):
d[l] += x;
d[r + 1] -= x;
// finalize once:
vector<ll> a(n);
ll run = 0;
for (int i = 0; i < n; i++) { run += d[i]; a[i] = run; }
```

- Mental model: d is the discrete derivative of a ; a range-add changes the derivative only at the two endpoints; the final prefix sum is the integral. (If updates and reads interleave, this fails — that's segment-tree territory, Tier 3.)

1.10 Brute force and exhaustive enumeration

Triggers. The constraints table (§0.4) says you can afford it — $n \leq 20$ screams subsets, $n \leq 10$ screams permutations. Also: brute force is your stress-test reference (§0.3) even when the real solution is clever.

```
// all subsets of n items via bitmask (n <= ~24)
for (int mask = 0; mask < (1 << n); mask++) {
    ll sum = 0;
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) sum += a[i];    // item i is in this subset
    // process subset 'mask'
}

// all permutations (n <= ~10-11)
vector<int> p(n); iota(p.begin(), p.end(), 0);
do {
    // process permutation p
} while (next_permutation(p.begin(), p.end()));
```

- `next_permutation` enumerates in lexicographic order starting from the current state — sort first to get all of them.
- Half the value of brute force is psychological: write it first when stuck. It clarifies the problem statement, gives you a checker, and sometimes reveals the pattern that *is* the intended solution.

1.11 Recursion

Mechanics you must control: every call needs a shrinking argument and a base case; state passed by reference (`vector<int>&`) to avoid copying. The idiomatic CP way to write a recursive function that captures local state is a self-referencing lambda:

```
// portable since C++14: pass the lambda to itself
auto dfs = [&](auto&& self, int u, int par) -> void {
    for (int v : g[u]) if (v != par) self(self, v, u);
};
dfs(dfs, 0, -1);

// C++23 "deducing this" (needs GCC 14+) -- cleaner, no self-passing:
auto dfs = [&](this auto&& self, int u, int par) -> void {
    for (int v : g[u]) if (v != par) self(v, u);
};
dfs(0, -1);
```

- Always write the explicit `-> void` (or return type): recursive lambdas cannot deduce their own return type.
- Depth budget: recursion to $\sim 10^6$ frames is generally fine on Codeforces (large stack); locally you may need `ulimit -s unlimited`. A bamboo-shaped tree (a path) reaches depth n — if n is 10^6 and your frames are fat with local arrays, convert to an explicit stack.

1.12 Backtracking

Triggers. "Generate / count / find any" over combinatorial objects with constraints (placements, assignments, sequences), small constraints, and the ability to **prune**: detect early that a partial choice can never complete. Backtracking = DFS over partial solutions.

```
// universal skeleton
void rec(int i) {
    // i = next decision index
    if (dead_end()) return; // PRUNE: partial state already invalid
    if (i == n) { record_solution(); return; }
    for (auto choice : options(i)) {
        apply(choice);
        rec(i + 1);
        undo(choice); // restore state -- the heart of backtracking
    }
}

// n-queens: one queen per row; track attacked columns/diagonals
int n, cnt = 0;
vector<bool> col, d1, d2; // sizes n, 2n, 2n
void rec(int r) {
    if (r == n) { cnt++; return; }
    for (int c = 0; c < n; c++) {
        if (col[c] || d1[r + c] || d2[r - c + n]) continue;
        col[c] = d1[r + c] = d2[r - c + n] = true;
        rec(r + 1);
        col[c] = d1[r + c] = d2[r - c + n] = false;
    }
}
```

- Every mutation must have a matching undo on *every* exit path — a missed undo after an early return is the classic backtracking bug. Structure the code so `apply/undo` wrap the recursive call and nothing else.
- Order options so failures happen early (most-constrained first) — pruning power is exponential.

1.13 Greedy basics

Triggers. "Maximize the number of...", "minimum cost if you process in some order", choices that don't interact beyond a simple resource. Tier-1 greedy = sort by the right key + take while feasible.

```
// interval scheduling: max number of non-overlapping intervals
ranges::sort(seg, {}, &pair<ll, ll>::second); // by RIGHT endpoint
int cnt = 0; ll last = LLONG_MIN;
for (auto [l, r] : seg)
    if (l >= last) { cnt++; last = r; } // touching allowed; use l > last otherwise
```

- Why right endpoint (one-line exchange argument): among intervals you could pick first, the one ending earliest leaves the most room — any optimal solution's first interval can be swapped for it without breaking anything. That swap-without-loss reasoning is how every greedy is justified; at Tier 1 just verify it informally, but do verify: the second most common wrong answer at 1200+ is a plausible greedy with a hidden counterexample.
- Standard keys worth memorizing: earliest finish (interval scheduling), earliest deadline (scheduling with deadlines), largest value first (when items don't conflict). When you can't prove the key, stress-test it (§0.3) before trusting it.

1.14 Divide and conquer basics

Triggers. The problem combines cleanly from two halves; "count pairs (i, j) with i in left half, j in right". $T(n) = 2T(n/2) + O(n) = O(n \log n)$. Canonical example — counting inversions while merge-sorting:

```
ll inversions(vector<int>& a, int l, int r) { // half-open [l, r)
    if (r - l <= 1) return 0;
    int m = (l + r) / 2;
    ll res = inversions(a, l, m) + inversions(a, m, r);
    // count cross pairs a[i] > a[j], i in [l, m), j in [m, r): both halves sorted
    vector<int> tmp; tmp.reserve(r - l);
    int i = l, j = m;
    while (i < m || j < r) {
        if (j == r || (i < m && a[i] <= a[j])) tmp.push_back(a[i++]);
        else { res += m - i; tmp.push_back(a[j++]); } // a[j] beats all remaining left
    }
    copy(tmp.begin(), tmp.end(), a.begin() + l);
    return res;
}
```

- The reusable insight: "solve halves, then handle interactions *across* the cut in linear time". Many harder problems are exactly this with a fancier cross step.

1.15 Modular arithmetic

Triggers. "Output the answer modulo 10^9+7 / 998244353" — i.e., the real answer is astronomically large and the problem is about counting. Every +, −, × in your solution must be done mod m.

```
const ll MOD = 1'000'000'007; // 998'244'353 in FFT-flavored problems

ll add(ll a, ll b) { a += b; return a >= MOD ? a - MOD : a; }
ll sub(ll a, ll b) { a -= b; return a < 0 ? a + MOD : a; }
ll mul(ll a, ll b) { return a * b % MOD; } // safe: both < MOD < 2^30, product < 2^60
```

- **The sign trap:** in C++, $(-7) \% 3 == -1$, not 2. Any subtraction needs $(x \% MOD + MOD) \% MOD$ or the sub above. This is the single most frequent modular bug.
- Keep every stored value already reduced to $[0, MOD)$; then plain $a * b \% MOD$ in ll never overflows ($10^9 \cdot 10^9 < 9.2 \cdot 10^{18}$). Overflow appears only when you multiply unreduced values.
- Division does not exist mod m — "dividing by a" means multiplying by a's modular inverse (§2.21). Flag every division in your formula as future work for that section.

1.16 GCD and LCM

```

ll g = gcd(a, b); // <numeric>, C++17 -- or own Euclid:
ll my_gcd(ll a, ll b) { while (b) { a %= b; swap(a, b); } return a; }
ll l = a / gcd(a, b) * b; // divide FIRST -- a*b may overflow before /g

```

- Euclid's step in one sentence: any common divisor of (a, b) also divides a mod b, and vice versa — so the pair (b, a mod b) has exactly the same divisors. Runtime $O(\log \min(a, b))$ (worst case: Fibonacci numbers).
- $\text{gcd}(x, 0) == x$ — handy as a fold identity: `accumulate(v.begin(), v.end(), 0LL, gcd<ll, ll>)` or just loop. gcd of a range only ever decreases or stays — useful structural fact.

1.17 Binary exponentiation

Triggers. $a^e \bmod m$ with e up to 10^{18} ; modular inverse via Fermat (§2.21); later, matrix powers. Idea: square the base, halve the exponent — uses the binary digits of e, $O(\log e)$ multiplications.

```

ll power(ll b, ll e, ll mod) {
    ll r = 1; b %= mod;
    while (e > 0) {
        if (e & 1) r = r * b % mod; // this binary digit of e is set
        b = b * b % mod;           // b^1 -> b^2 -> b^4 -> ...
        e >>= 1;
    }
    return r;
}

```

- Memorize this exact loop; it reappears unchanged for matrices and other associative operations.

1.18 Sieve of Eratosthenes (+ smallest-prime-factor sieve)

Triggers. Primality / factorization for **many** numbers up to $\sim 10^7$, "for every $i \leq n$: something about its prime structure".

```

// classic: prime flags up to N in  $O(N \log \log N)$ 
vector<bool> comp(N + 1, false);
vector<int> primes;
for (int i = 2; i <= N; i++) {
    if (!comp[i]) primes.push_back(i);
    if (!comp[i])
        for (ll j = (ll)i * i; j <= N; j += i) comp[j] = true; // start at  $i^2$ 
}

// SPF sieve: smallest prime factor of every number -> factor anything in  $O(\log x)$ 
vector<int> spf(N + 1, 0);
for (int i = 2; i <= N; i++)
    if (spf[i] == 0)
        for (int j = i; j <= N; j += i)
            if (spf[j] == 0) spf[j] = i;

auto factor = [&](int x) { // x <= N
    vector<pair<int, int>> f;
    while (x > 1) {
        int p = spf[x], e = 0;
        while (x % p == 0) x /= p, e++;
        f.push_back({p, e});
    }
    return f;
};

```

- Start crossing out at i^2 (everything smaller with factor i was already hit by a smaller prime) — and cast to ll there: i^2 overflows int for $i > \sim 46000$.
- The SPF variant is the workhorse: build once, then factor 10^5 queries instantly. Prefer it whenever multiple factorizations are needed.

1.19 Trial division

Triggers. Factor or primality-check a **single** number up to $\sim 10^{12}$ – 10^{14} ($\sqrt{n} \leq 10^6$ – 10^7 loop iterations).

```
vector<pair<ll,int>> factor(ll n) {
    vector<pair<ll,int>> f;
    for (ll d = 2; d * d <= n; d++)
        if (n % d == 0) {
            int e = 0;
            while (n % d == 0) n /= d, e++;
            f.push_back({d, e});
        }
    if (n > 1) f.push_back({n, 1});    // leftover is a single prime -- DON'T forget it
    return f;
}
```

- Why $\sqrt{n}$ suffices: a composite n has some factor $\leq \sqrt{n}$ (if both factors were larger, their product would exceed n). The leftover after the loop is therefore prime.
- Forgetting the final `if (n > 1)` is the canonical bug — your factorization of $2 \cdot 10^9 + 33$ (a prime times 1) silently comes back empty.

PART 2 — TIER 2 (CF ~1200-1700): techniques with intuition

From here on, every section adds an **Intuition** block (shaded green): the mental picture, the reason the method is correct, and the reason for its complexity. Read the code *after* the intuition — the code should then feel inevitable rather than memorized.

2.1 Graph representation

Triggers. The statement says "cities and roads", "friends", "grid moves" — or doesn't: many problems are secretly graphs where **states are nodes and legal moves are edges** (a word ladder, a puzzle position, a (row, col) cell). Spotting the hidden graph is the recurring Tier-2 skill.

```
int n, m; cin >> n >> m;
vector<vector<int>> g(n); // adjacency list -- the default
for (int i = 0; i < m; i++) {
    int u, v; cin >> u >> v; --u; --v; // 1-indexed input -> 0-indexed
    g[u].push_back(v);
    g[v].push_back(u); // delete for directed graphs
}
vector<vector<pair<int,int>>> gw(n); // weighted: (neighbor, weight)
vector<array<int,3>> edges; // edge list: for DSU / sorting edges
```

- Adjacency list: $O(n + m)$ memory, perfect for traversal — use it 95% of the time. Edge list: when you process *edges* as objects (sort by weight, DSU). Adjacency *matrix* only when $n \leq \sim 2000$ and you need $O(1)$ "is there an edge".
- Convert to 0-indexed at the moment of reading and never think about it again. Mixed indexing is a leading source of silent wrong answers.

Intuition. A graph is just "things + binary relation". The representation question is only: do you iterate *neighbors of a node* (list), *all relations* (edge list), or *query a specific pair* (matrix)? Choose by access pattern, nothing else.

2.2 BFS — breadth-first search

Triggers. "Minimum number of moves/steps/edges", shortest path where every step costs the same, "can X reach Y", level-by-level processing, shortest transformation sequences (states = nodes).

Intuition. Drop a stone in water: BFS is the expanding ripple. The queue always holds the current frontier — first every node at distance d , then every node at distance $d+1$. The invariant: distances inside the queue are non-decreasing and span at most two consecutive values. Therefore the *first* time you reach a node, you reached it along a shortest route — there is no way a later, shorter route exists, because everything later in the queue is at least as far. That is the entire correctness proof, and it is also why BFS breaks the moment edges have different weights (a "cheap long" path can arrive after an "expensive short" one — Dijkstra, Tier 3, fixes exactly this).

```
vector<int> dist(n, -1), par(n, -1); // dist doubles as 'visited'
queue<int> q;
dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) if (dist[v] == -1) {
        dist[v] = dist[u] + 1;
        par[v] = u; // for path reconstruction
        q.push(v);
    }
}
// reconstruct s->t: follow par[] from t backwards, then reverse

// grid BFS -- the same algorithm, neighbors generated on the fly
const int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1};
vector dist(R, vector<int>(C, -1));
queue<pair<int,int>> q;
dist[sr][sc] = 0; q.push({sr, sc});
```

```

while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int k = 0; k < 4; k++) {
        int nr = r + dx[k], nc = c + dy[k];
        if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
        if (grid[nr][nc] == '#' || dist[nr][nc] != -1) continue;
        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}

```

- **Mark when pushing, not when popping.** Marking on pop lets the same node enter the queue many times — correct answers, blown-up memory and time.
- Adaptations: several starting points → push them all with distance 0 before the loop ("multi-source", formally Tier 3 but it is literally this one change). Need the path → keep `par[]`. Level-order processing → consume the queue in size-batches (`int sz = q.size()` per layer).
- State-space BFS: the node can be anything hashable — (position, keys collected), (a, b) jug contents. Encode the state, put distances in a map or a multi-dimensional array. If the statement says "minimum operations", try this before anything clever.

2.3 DFS — depth-first search, timestamps, edge types

Triggers. Anything structural: explore everything reachable, detect cycles, order by dependencies, subtree computations, "is u an ancestor of v". DFS is less about distance and all about **structure**.

Intuition. A mouse in a maze with a ball of string: go as deep as possible, back up only when stuck. The recursion stack at any moment is exactly the path from the root to the current node — that single fact powers cycle detection (§2.6) and much more. Record entry time `tin[u]` and exit time `tout[u]`: because calls nest like parentheses, the interval `[tin[u], tout[u]]` of a node *contains* the intervals of exactly its descendants. "Is u an ancestor of v?" becomes two integer comparisons, and "u's subtree" becomes a contiguous range — the bridge to flattening trees onto arrays at higher tiers.

```

int timer = 0;
vector<int> tin(n), tout(n);
vector<bool> vis(n, false);
auto dfs = [&](auto&& self, int u) -> void {
    vis[u] = true;
    tin[u] = timer++;
    for (int v : g[u]) if (!vis[v]) self(self, v);
    tout[u] = timer++;
};
// ancestor test: u is an ancestor of v <=> tin[u] <= tin[v] && tout[v] <= tout[u]

```

- Edge classification while DFS-ing a **directed** graph: an edge `u→v` is a *tree edge* (v unvisited), a *back edge* (v is on the current stack — this means a cycle), or a forward/cross edge (v finished). In an **undirected** graph only tree and back edges exist — there is nothing "sideways" to jump to that wasn't already explored from below.
- On trees, pass the parent instead of a visited array: `for (v : g[u]) if (v != par) ...` — less state, and it generalizes to "do something on the way down, combine results on the way up", the seed of tree DP.

2.4 Connected components and flood fill

Triggers. "How many groups/islands/regions", "are a and b in the same group", anything that partitions by reachability in an undirected graph.

```

vector<int> comp(n, -1);
int C = 0;
for (int s = 0; s < n; s++) if (comp[s] == -1) {
    // BFS (or DFS) labeling everything reachable from s
    queue<int> q; q.push(s); comp[s] = C;
    while (!q.empty()) {

```

```

        int u = q.front(); q.pop();
        for (int v : g[u]) if (comp[v] == -1) { comp[v] = C; q.push(v); }
    }
    C++;
}
// comp[u]: component id; C: number of components
// flood fill = the identical loop over grid cells with 4- (or 8-) neighborhoods

```

Intuition. Reachability in an undirected graph is an equivalence relation, so it partitions nodes into classes — the loop just discovers one class per unvisited seed. Total cost $O(n + m)$: each node and edge is touched once across *all* the BFS runs combined, because the visited array is shared. Component sizes, per-component sums, "largest island" — all are one extra accumulator inside the same loop.

2.5 Bipartite check (2-coloring)

Triggers. "Split into two groups so that every conflict/edge crosses groups", "alternating" structures, two-state assignments along edges (later: 2-SAT, matchings build on this).

Intuition. Walk along any edge: the color must flip. So within one component, choosing the seed's color forces every other node's color — along every path. A node reachable by an even-length and an odd-length path receives two contradictory demands; that pair of paths closes an odd cycle. Hence the theorem worth keeping: **bipartite $\iff$ no odd cycle**. The algorithm is just BFS executing the forcing and reporting the first contradiction.

```

vector<int> col(n, -1);
bool ok = true;
for (int s = 0; s < n && ok; s++) if (col[s] == -1) {
    col[s] = 0;
    queue<int> q; q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (col[v] == -1) { col[v] = col[u] ^ 1; q.push(v); }
            else if (col[v] == col[u]) { ok = false; }
        }
    }
}

```

- Run from every unvisited node — a disconnected graph is bipartite only if every component is.
- Adaptation seen constantly at 1300–1600: edges given as "u and v must differ" → bipartite check; "u and v must be equal" too → put equal-pairs in one DSU group first or model both relations (preview of weighted DSU, Tier 3).

2.6 Cycle detection

Triggers. "Is it a tree?" (connected + $n-1$ edges + no cycle), "can the dependencies be satisfied", "process forever or terminate".

Undirected: during DFS, meeting a visited neighbor that is not the parent closes a cycle. Or simpler: add edges into a DSU (§2.8); an edge joining two already-connected nodes is the cycle.

```

bool cycle = false;
auto dfs = [&](auto&& self, int u, int par) -> void {
    vis[u] = true;
    for (int v : g[u]) {
        if (v == par) { par = -2; continue; } // skip the parent edge ONCE (multi-edge safe-ish)
        if (vis[v]) cycle = true;
        else self(self, v, u);
    }
};

```

Directed: three colors. White = untouched, gray = on the current recursion stack, black = finished. An edge into a **gray** node is a back edge — a cycle. An edge into a black node is harmless.

```

vector<int> state(n, 0); // 0 white, 1 gray, 2 black
bool cycle = false;
auto dfs = [&](auto&& self, int u) -> void {
    state[u] = 1;
    for (int v : g[u]) {
        if (state[v] == 0) self(self, v);
        else if (state[v] == 1) cycle = true; // back edge -> cycle
    }
    state[u] = 2;
};
for (int i = 0; i < n; i++) if (state[i] == 0) dfs(dfs, i);

```

Intuition. Gray nodes are precisely your ancestors in the DFS tree — the path you walked in on. An edge back into that path plus the path itself is the cycle, visibly. Why "plain visited" fails for directed graphs: a visited node might be in a completely different, already-finished branch (a cross edge), which closes no cycle. The gray/black distinction is exactly "still on my path" vs "someone else's finished business". For undirected graphs the parent check replaces all of this, with one subtlety: a doubled edge $u-v$ IS a cycle, so skip the parent only once (track the edge id, or the flag trick above).

- Alternative for directed graphs: run Kahn's algorithm (§2.7) — if fewer than n nodes get ordered, the leftovers contain the cycle. Often the more convenient tool when you needed the ordering anyway.

2.7 Topological sort

Triggers. Dependencies: "task A before task B", course prerequisites, build orders; also the processing order for any DP on a DAG. Only DAGs (directed, acyclic) have one.

Variant 1 — DFS: append each node when it *finishes*; reverse at the end.

```

vector<int> order, state(n, 0);
auto dfs = [&](auto&& self, int u) -> void {
    state[u] = 1;
    for (int v : g[u]) if (state[v] == 0) self(self, v);
    state[u] = 2;
    order.push_back(u); // u finishes only after all its out-neighbors
};
for (int i = 0; i < n; i++) if (state[i] == 0) dfs(dfs, i);
reverse(order.begin(), order.end());

```

Variant 2 — Kahn's (BFS on indegrees): repeatedly emit nodes with indegree 0.

```

vector<int> indeg(n, 0);
for (int u = 0; u < n; u++) for (int v : g[u]) indeg[v]++;
queue<int> q;
for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
vector<int> order;
while (!q.empty()) {
    int u = q.front(); q.pop();
    order.push_back(u);
    for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
}
// order.size() < n <=> the graph has a cycle

```

Intuition. DFS version: a node is appended only after every node it points to has already been appended — so in the *reversed* list, every edge goes left → right. That's the definition of a topological order. Kahn's version: a node with no incoming edges depends on nothing, so it can safely go first; removing it can only create new dependency-free nodes. Think of it as peeling the DAG from its sources. If peeling stalls before n nodes, every remaining node has an incoming edge from another remaining node — following those backwards forever inside a finite set must revisit a node: a cycle. So Kahn's detects cycles for free.

- Need the lexicographically smallest valid order? Kahn's with a `priority_queue` instead of a queue. (The DFS variant can't do this.)

2.8 DSU — disjoint set union (union-find)

Triggers. Dynamic connectivity where edges are only **added**: "after each road is built, are X and Y connected / how many provinces remain"; grouping by equivalence ("a equals b" facts); Kruskal's MST later. If edges get *removed*, plain DSU does not apply (offline tricks exist at higher tiers).

Intuition. Every group elects a representative ("the boss"); each member stores a pointer toward the boss, forming a tree. `find(x)` walks up to the boss; union points one boss at the other. Two refinements make it near- $O(1)$. **Path compression:** once you've walked the chain and learned who the boss is, re-pin everyone on the chain directly to the boss — you pay once so all future queries pay almost nothing (classic amortization: expensive operations make their own future cheap). **Union by size:** always attach the smaller tree under the larger. Then any element's distance to its boss grows only when its whole group gets absorbed into a bigger one — which at least doubles the group's size, so it can happen at most $\log n$ times per element. Together the amortized cost is $\alpha(n)$, the inverse Ackermann function — below 5 for any input that fits in this universe; treat it as constant.

```
struct DSU {
    vector<int> p, sz;
    int comps;
    DSU(int n) : p(n), sz(n, 1), comps(n) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); } // path compression
    bool same(int a, int b) { return find(a) == find(b); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false; // already together -- this edge closes a cycle
        if (sz[a] < sz[b]) swap(a, b); // union by size
        p[b] = a; sz[a] += sz[b]; comps--;
        return true;
    }
};
```

- The false return of `unite` is itself a tool: it detects cycles while building (§2.6) and is the core of Kruskal's MST (Tier 3).
- Store any per-group aggregate at the representative (size is built in; sums, min label, counts all work the same way): merge the aggregate when merging the trees.
- Pitfall: comparing `p[a] == p[b]` instead of `find(a) == find(b)` — parents are not representatives mid-structure.

2.9 Monotonic stack

Triggers. "For each element, the nearest greater/smaller element to the left/right"; spans ("how far back does this stay the max"); largest rectangle in a histogram; "next warmer day". Whenever each element needs its closest dominating neighbor, in $O(n)$ total.

Intuition. People standing in a line, all looking left at heights. When a new person arrives, anyone shorter (for a nearest-*greater* query) standing before them becomes permanently invisible to everyone arriving later — the newcomer is both closer and taller, dominating them as an answer forever. So discard them: `pop`. What remains on the stack is exactly the "skyline" — a monotone sequence of still-relevant candidates, and your answer is whoever survives on top. Complexity is the amortized classic: every element is pushed once and popped at most once, so the nested-looking while loop is $O(n)$ total. The invariant to hold onto: **the stack is always sorted**, and everything ever removed was provably useless for the entire future.

```
// prevSmaller[i] = index of nearest j < i with a[j] < a[i], else -1
vector<int> st, prevSmaller(n, -1);
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
    if (!st.empty()) prevSmaller[i] = st.back();
    st.push_back(i);
}
// nextSmaller: same loop from the right
```

```
// nearest GREATER: flip the comparison to <=
// largest rectangle in a histogram, O(n)
// bar i can stretch over (prevSmaller[i], nextSmaller[i]) exclusive
ll best = 0;
for (int i = 0; i < n; i++) {
    int L = prevSmaller[i], R = nextSmaller[i];    // -1 and n act as walls
    best = max(best, (ll)a[i] * (R - L - 1));
}
```

- Strict vs non-strict ($\geq$ vs $>$ in the pop) decides how ties behave — derive it per problem from "who may dominate whom"; with equal heights, exactly one side should treat equals as blocking or you'll double-count rectangles.
- Store **indices**, not values — you almost always need positions (for widths/spans), and values are a lookup away.

2.10 Monotonic deque — sliding window min/max

Triggers. "Maximum/minimum of every window of size k " in $O(n)$; later, the same structure speeds up DPs whose transition is "best over the last k states".

Intuition. Keep, inside the current window, only the elements that could still become the maximum of *some future* window. An element that is older AND not larger than a newer one is dominated for the rest of time — drop it from the back. What survives is a list decreasing in value and increasing in age... actually increasing in index — so the front is simultaneously the oldest survivor and the largest: the answer. The front retires only when it physically slides out of the window. Same push-once/pop-once accounting as the monotonic stack: $O(n)$ total. A deque rather than a stack purely because elements leave from both ends — domination kills at the back, expiry kills at the front.

```
// maximum of every length-k window
deque<int> dq;                // indices; a[dq] strictly decreasing
vector<int> out;
for (int i = 0; i < n; i++) {
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // dominated
    dq.push_back(i);
    if (dq.front() <= i - k) dq.pop_front();                  // expired
    if (i >= k - 1) out.push_back(a[dq.front()]);
}
```

- Minimum: flip the comparison. Both at once: two deques.
- Sanity check that catches most implementation slips: with $k = 1$ the output must equal the array; with $k = n$ it must be a single global max.

2.11 Thinking in DP — read this before any DP section

Triggers for DP in general. "Count the ways", "maximum/minimum over sequences of decisions", "is it possible", combined with choices at each step and constraints that only depend on a small summary of the past. Constraints like $n \leq 5000$ ($O(n^2)$ states) or weights $\leq 10^5$ are strong hints.

Intuition. DP is brute force with deduplication. Imagine the full recursion tree of "try every decision": it is exponentially large, but huge numbers of its nodes are *the same subproblem reached by different histories*. The key act of DP design is choosing a **state**: the minimal summary of the past that the future actually needs. If two histories lead to the same state, the future treats them identically — so solve each state once. The states plus their transitions form a DAG; DP is just evaluating that DAG in dependency order (which is why topological thinking, §2.7, reappears). A proposed state is correct exactly when the *optimal substructure* check passes: knowing only the state (not the history), can you still enumerate the next decisions and their effects? If some forgotten detail of history matters, the detail belongs in the state.

The design recipe — apply it mechanically until it becomes instinct:

- 1. Define the state in words, as the answer to a restricted problem: "dp[i] = best score using the first i items", "dp[i][j] = number of ways to ... considering prefix i with j of something". Writing the sentence is not optional; fuzzy states produce wrong transitions.
- 2. Find the transition by asking "what was the **last** decision?" and removing it — that exposes a smaller state. Sum over the possibilities (counting) or take min/max (optimization).
- 3. Base cases = the smallest states where the answer is known by definition (often dp[0] = 1 for "empty prefix, one way: do nothing").
- 4. Order: bottom-up loops following the DAG, or top-down recursion with memoization — identical results; bottom-up is usually faster and easier to optimize, top-down is easier to get correct first. Know both.

```
// the two shapes, on Fibonacci as a stand-in
// top-down (memoization)
vector<ll> memo(n + 1, -1);
auto f = [&](auto&& self, int i) -> ll {
    if (i <= 1) return i;
    ll& r = memo[i];
    if (r != -1) return r;
    return r = self(self, i - 1) + self(self, i - 2);
};

// bottom-up (tabulation)
vector<ll> dp(n + 1);
dp[0] = 0; dp[1] = 1;
for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
```

- Complexity = (#states) × (work per transition). Before coding, compute it and check against §0.4 — if it doesn't fit, the state is wrong (too rich) or an optimization (§2.19 and beyond) is intended.
- Debugging DP: print the table for a tiny input and recompute two or three cells by hand; stress-test against brute force (§0.3). DP bugs are almost always a wrong base case, a wrong loop order, or an off-by-one in the state's meaning.

2.12 Linear DP

Triggers. Decisions march left-to-right over a sequence; each position's choice depends on O(1) or O(n) previous positions. The bread-and-butter shape of 1200–1500 problems.

```
// max-sum contiguous subarray (Kadane) -- dp = best sum of a subarray ENDING here
ll best = a[0], cur = a[0];
for (int i = 1; i < n; i++) {
    cur = max((ll)a[i], cur + a[i]); // extend the previous run, or start fresh
    best = max(best, cur);
}

// "house robber": max sum of non-adjacent elements
// dp[i][0/1] = best over prefix i, last element skipped/taken
ll skip = 0, take = a[0];
for (int i = 1; i < n; i++) {
    ll nskip = max(skip, take); // free to do anything before a skipped i
    ll ntake = skip + a[i];     // taking i forces i-1 skipped
    skip = nskip; take = ntake;
}
ll ans = max(skip, take);
```

Intuition. Both follow the recipe of §2.11. Kadane's state is "best subarray **ending at i**" — the word "ending" is the whole trick: it makes the transition local (extend or restart), and the global answer is the best over all endings. Adding a tag like ending here / last one taken / currently inside a block is the standard repair when a naive state forgets something the future needs. Notice also the memory trick: when dp[i] depends only on dp[i-1], keep two scalars instead of an array — the same idea scales to keeping two rows of a 2D table.

- Recognize the family: "delete/keep each element under adjacency rules", "min cost to reach the end with jumps of 1-2", "count binary strings without two adjacent ones" — all linear DP with 1-3 tagged states per index.

2.13 Knapsacks — 0/1, unbounded, bounded

Triggers. Items with weight/cost and value, a capacity, choose a subset (or multiset) to optimize. Also in disguise: "can some subset sum to S" (values = booleans), "split into two halves as equal as possible".

```
// 0/1 knapsack: each item at most once. dp[c] = best value at capacity exactly <= c
vector<ll> dp(W + 1, 0);
for (auto [w, val] : items)
    for (int c = W; c >= w; c--) // DOWNWARD -- see intuition
        dp[c] = max(dp[c], dp[c - w] + val);

// unbounded knapsack: unlimited copies -- only the loop direction changes
for (auto [w, val] : items)
    for (int c = w; c <= W; c++) // UPWARD: reuse on purpose
        dp[c] = max(dp[c], dp[c - w] + val);

// subset-sum feasibility (0/1, values irrelevant)
vector<bool> can(W + 1, false); can[0] = true;
for (int w : weights)
    for (int c = W; c >= w; c--)
        can[c] = can[c] || can[c - w];
```

Intuition. The honest table is $dp[i][c]$ = best using the first i items at capacity c ; each row depends only on the previous row, so we keep one row. Now the loop direction is everything. Going **downward**, $dp[c-w]$ still holds the *previous item's* row — the current item contributes at most once. Going **upward**, $dp[c-w]$ may already include the current item, so it gets stacked repeatedly — which is precisely the unbounded semantics. One character of code encodes "use once" versus "use many". If you ever doubt a direction, re-expand to the 2D table mentally and ask: does $dp[c-w]$ mean row $i-1$ or row i ?

Bounded knapsack (item i available k_i times): split k_i into binary bundles 1, 2, 4, ..., remainder, and run 0/1 on the bundles — any count $0..k_i$ is expressible as a subset of bundles, and the item count only grows by a log factor.

```
// bounded -> 0/1 via binary splitting
vector<pair<int, ll>> bundle; // (weight, value)
for (auto [w, val, k] : items)
    for (int take = 1; k > 0; take *= 2) {
        int t = min(take, k); k -= t;
        bundle.push_back({w * t, val * (ll)t});
    }
// then plain 0/1 over 'bundle'
```

- Counting versions: replace `max` with addition mod p , with $dp[0] = 1$. The 0/1-counting version answers "number of subsets with sum c ".
- Complexity $O(n \cdot W)$ — pseudo-polynomial: fine for $W \leq \sim 10^7/n$, the reason knapsack problems always print a small capacity bound. If W is 10^9 , knapsack is NOT the intended solution.

2.14 Coin change — and the ordered/unordered subtlety

Triggers. "Fewest coins for amount A ", "how many ways to pay A ". The counting version hides one of the most instructive traps in beginner DP.

```
// minimum number of coins (unbounded)
vector<int> dp(A + 1, INT_MAX / 2); dp[0] = 0;
for (int x = 1; x <= A; x++)
    for (int c : coins) if (c <= x)
        dp[x] = min(dp[x], dp[x - c] + 1);
```

```
// count UNORDERED ways (combinations): {1,2} == {2,1}
vector<ll> ways(A + 1, 0); ways[0] = 1;
for (int c : coins) // coins OUTER
    for (int x = c; x <= A; x++)
        ways[x] = (ways[x] + ways[x - c]) % MOD;

// count ORDERED ways (sequences/compositions): 1+2 and 2+1 are different
vector<ll> seq(A + 1, 0); seq[0] = 1;
for (int x = 1; x <= A; x++) // amount OUTER
    for (int c : coins) if (c <= x)
        seq[x] = (seq[x] + seq[x - c]) % MOD;
```

Intuition. Why does swapping two loops change *what is being counted*? With coins outermost you commit to a canonical order: first decide how many pennies, then how many nickels, ... — every multiset of coins is assembled in exactly one way, so multisets are counted once. With the amount outermost, every position of the sequence freely picks any coin — you are counting ordered sequences that sum to A. Neither is "the" coin change; read the statement and decide whether {1,2} and {2,1} are the same payment. This loop-order-defines-the-object lesson generalizes: in any counting DP, the iteration order is part of the combinatorial definition.

- The min-coins version doesn't care about order (a minimum is a minimum), so either loop order works there.

2.15 Longest increasing subsequence (LIS)

Triggers. "Longest chain" under an order; delete fewest to make increasing; box/envelope nesting (sort by one dimension, LIS the other). Subsequence $\neq$ substring: elements need not be contiguous.

```
// O(n^2): dp[i] = LIS ending exactly at i
vector<int> dp(n, 1);
int ans = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++)
        if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
    ans = max(ans, dp[i]);
}

// O(n log n) -- learn it now, you will need it by 1400:
// tail[k] = smallest possible last element of an increasing subsequence of length k+1
vector<int> tail;
for (int x : a) {
    auto it = lower_bound(tail.begin(), tail.end(), x); // upper_bound -> non-decreasing LIS
    if (it == tail.end()) tail.push_back(x);
    else *it = x;
}
int ans = tail.size();
```

Intuition. The $O(n^2)$ state is the familiar "...ending at i" tag. The $O(n \log n)$ version rests on one exchange-style insight: between two increasing subsequences of the same length, the one with the **smaller tail** is at least as extendable — strictly more numbers can follow it. So for each length, only the minimal tail matters. These tails are automatically sorted (a length-5 chain contains a length-4 chain with a smaller tail inside it), which is what legalizes the binary search. A new element x improves exactly one entry: the first tail $\geq x$ (replacing it keeps that length's chain valid and lowers its tail), or extends the table if x beats every tail. Note tail[] is NOT an actual subsequence — it's a table of best tails per length; the *length* is the answer.

- Variants: non-decreasing $\rightarrow$ upper_bound; longest *decreasing* $\rightarrow$ negate values; need the actual subsequence $\rightarrow$ store predecessor indices in the $O(n^2)$ version (simplest at this level).

2.16 Longest common subsequence (LCS)

Triggers. Two sequences, "longest shared subsequence", minimum edits to transform (edit distance is the same table with different transitions), diff-like comparisons.

```

vector dp(n + 1, vector<int>(m + 1, 0)); // dp[i][j]: LCS of a[0..i) and b[0..j)
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        dp[i][j] = (a[i-1] == b[j-1]) ? dp[i-1][j-1] + 1
                                   : max(dp[i-1][j], dp[i][j-1]);

int ans = dp[n][m];

// reconstruction: walk back from (n, m)
string res;
for (int i = n, j = m; i > 0 && j > 0; ) {
    if (a[i-1] == b[j-1]) { res += a[i-1]; i--; j--; }
    else if (dp[i-1][j] >= dp[i][j-1]) i--;
    else j--;
}
reverse(res.begin(), res.end());

```

Intuition. Look at the **last** characters of both prefixes — the §2.11 "last decision" move. If they're equal, some optimal LCS matches them (if an optimal solution didn't, you could re-match the final equal pair without losing length — a mini exchange argument), so take them and recurse on both shorter prefixes. If they differ, they can't both end the LCS, so at least one is droppable: take the better of dropping either. Two prefix lengths fully summarize the past → $O(nm)$ states, $O(1)$ transition. Edit distance, "minimum insertions to make a palindrome" (LCS with the reverse), and counting common subsequences are all this table with the cells reinterpreted.

2.17 Grid DP

Triggers. Paths in a grid moving right/down (or any acyclic move set): count paths, min/max path sum, with obstacles. The 2D rehearsal of linear DP.

```

// number of paths from (0,0) to (R-1,C-1), moving right or down, '#' blocked
vector dp(R, vector<ll>(C, 0));
dp[0][0] = (grid[0][0] != '#');
for (int r = 0; r < R; r++)
    for (int c = 0; c < C; c++) {
        if (grid[r][c] == '#') { dp[r][c] = 0; continue; }
        if (r) dp[r][c] = (dp[r][c] + dp[r-1][c]) % MOD;
        if (c) dp[r][c] = (dp[r][c] + dp[r][c-1]) % MOD;
    }

```

Intuition. The move set "right/down" makes the grid a DAG whose topological order is plain row-major iteration — that's why the two nested loops just work. Every grid DP question is really: (1) what are the cells' dependencies, (2) does my loop order respect them? Min path sum = same loops with min and +cost; "down-left/down-right" moves (triangle problems) = same idea, different parents. When moves can go in all 4 directions the DAG property dies — that's no longer DP but BFS/Dijkstra territory; checking for cycles in the move set is the recognition step.

2.18 2D prefix sums and 2D difference arrays

Triggers. Many rectangle-sum queries on a static grid; many rectangle-add updates read once at the end; "count points/ones inside axis-aligned rectangles".

```

// build: P[i][j] = sum of the i x j top-left subgrid (1-based padding)
vector P(n + 1, vector<ll>(m + 1, 0));
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] - P[i][j];

// query: sum over rows [r1, r2) x cols [c1, c2) (half-open)
ll S = P[r2][c2] - P[r1][c2] - P[r2][c1] + P[r1][c1];

// 2D difference: add x on rows [r1, r2) x cols [c1, c2)
d[r1][c1] += x; d[r1][c2] -= x; d[r2][c1] -= x; d[r2][c2] += x;
// finalize: take the 2D prefix sum of d -- the result is the grid

```

Intuition. Pure inclusion-exclusion geometry. Building: the L-shape above-and-left double-counts the top-left block — subtract it back once. Querying: big rectangle minus the strip above minus the strip to the left re-removes their overlap twice — add the corner back. Draw the four rectangles once on paper and the signs become impossible to forget. The difference array is the same picture run backwards (it must be: "prefix-sum" and "difference" are inverse operators, as in 1D, §1.9). The 1-based padding row/column of zeros exists so $r1 = 0$ or $c1 = 0$ needs no special case.

2.19 DP with prefix-sum optimization

Triggers. A DP whose transition sums (or adds up counts over) a **contiguous range** of earlier states: $dp[i] = \sum dp[j]$ for j in $[i-k, i-1]$. Naively $O(n \cdot k)$ — the constraints (n, k up to 10^5+) tell you that's intended to die.

```
// example: ways to climb to step n with jumps of size 1..k
// dp[i] = sum of dp[i-k .. i-1]; pre[i] = dp[0] + ... + dp[i-1]
vector<ll> dp(n + 1), pre(n + 2);
dp[0] = 1; pre[1] = 1; // pre[0] = 0
for (int i = 1; i <= n; i++) {
    int lo = max(0, i - k);
    dp[i] = (pre[i] - pre[lo] + MOD) % MOD; // sum dp[lo..i-1] in O(1)
    pre[i + 1] = (pre[i] + dp[i]) % MOD;
}
```

Intuition. Nothing new — it's §1.8 applied to the DP table itself: maintain the running prefix of dp as you fill dp , and every range-sum transition collapses to two lookups. The general principle unlocked here, which scales all the way to Tier 4: stare at the transition's **shape**. Range-sum shape → prefix sums (this section). Range-min over a sliding window → monotonic deque (§2.10). Arbitrary range queries → Fenwick/segment tree (Tier 3). The DP doesn't change; only the data structure serving its transition does.

- Mind the mod: $pre[i] - pre[lo]$ can be negative after reduction — normalize. (§1.15's sign trap, again.)

2.20 Extended Euclidean algorithm

Triggers. Solve $a \cdot x + b \cdot y = c$ in integers ("pay exactly c with coins worth a and b "); modular inverse when the modulus is **not** prime; any time gcd alone isn't enough and you need the *certificate* coefficients.

```
// returns g = gcd(a, b) and fills x, y with a*x + b*y = g
ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

Intuition. Plain Euclid shrinks $(a, b) \rightarrow (b, a \bmod b)$ and the gcd survives. ExtGCD just carries the bookkeeping back up: suppose the recursive call found $b \cdot x_1 + (a \bmod b) \cdot y_1 = g$. Substitute the identity $a \bmod b = a - \lfloor a/b \rfloor \cdot b$ and regroup: $a \cdot y_1 + b \cdot (x_1 - \lfloor a/b \rfloor \cdot y_1) = g$. That regrouping IS the two assignment lines — the recursion performs the back-substitution you'd do by hand, one level at a time. Base case: $\gcd(a, 0) = a = a \cdot 1 + 0 \cdot 0$.

- Solvability: $a \cdot x + b \cdot y = c$ has integer solutions iff $g \mid c$ (scale the certificate by c/g). All solutions: $x = x_0 + (b/g) \cdot t$, $y = y_0 - (a/g) \cdot t$ — shifting mass between the two terms in the smallest step that keeps the sum fixed.
- $|x|, |y|$ stay manageable ($\leq \max(a, b)$ roughly), so ll is safe for inputs up to $\sim 10^{18}$.

2.21 Modular inverse

Triggers. Any division inside a mod-p computation: formulas with factorials/fractions, statements of the form "the answer is an irreducible fraction p/q ; print $p \cdot q^{-1} \bmod 998244353$ ". Division mod m = multiplication by the inverse.

```
// 1) prime modulus -- Fermat:  a^(MOD-2) is the inverse of a
ll inv(ll a) { return power(a, MOD - 2, MOD); }           // power from §1.17

// 2) any modulus with gcd(a, m) = 1 -- extended Euclid:
ll inv_general(ll a, ll m) {
    ll x, y;
    extgcd(a, m, x, y);           // a*x + m*y = 1  ->  a*x = 1 (mod m)
    return ((x % m) + m) % m;
}

// 3) all inverses 1..n in O(n) (handy for many divisions):
vector<ll> iv(n + 1); iv[1] = 1;
for (int i = 2; i <= n; i++) iv[i] = MOD - (MOD / i) * iv[MOD % i] % MOD;
```

Intuition. Why Fermat works: in the world mod a prime p , multiplying the nonzero residues $\{1, \dots, p-1\}$ by a just shuffles them ($a \cdot i \equiv a \cdot j$ would force $i \equiv j$, since a has no factor p to lose information with). Multiply everything in the shuffled set together: same product as before, but with an extra factor $a^{(p-1)}$. Cancel the common product: $a^{(p-1)} \equiv 1$. Peel one factor off: $a \cdot a^{(p-2)} \equiv 1$ — there's your inverse, computed by one binary exponentiation. The extGCD route is more honest about *when* inverses exist at all: $a \cdot x + m \cdot y = 1$ is solvable iff $\gcd(a, m) = 1$; read mod m , the $m \cdot y$ term vanishes and x is the inverse. No coprimality, no inverse — full stop.

- Trap: numbers $\equiv 0 \bmod p$ have no inverse. If a formula divides by something that can be a multiple of 10^9+7 (e.g., an input up to 10^{18}), the clean formula breaks on those inputs — handle separately.
- Don't recompute $\text{power}(a, \text{MOD}-2)$ inside hot loops (it's $O(\log \text{MOD})$ each); precompute what repeats (next section does exactly this for factorials).

2.22 Factorials, inverse factorials, nCr

Triggers. Counting problems — virtually all of them. Build the tables once per program, answer every $C(n, r)$ in $O(1)$.

```
const int N = 1'000'006;           // > max n you'll ever ask about
vector<ll> f(N), fi(N);
void build() {
    f[0] = 1;
    for (int i = 1; i < N; i++) f[i] = f[i-1] * i % MOD;
    fi[N-1] = power(f[N-1], MOD - 2, MOD); // ONE inverse...
    for (int i = N - 1; i >= 1; i--) fi[i-1] = fi[i] * i % MOD; // ...walked downward
}
ll C(ll n, ll r) {
    if (r < 0 || r > n) return 0;           // the guard that saves you in edge cases
    return f[n] * fi[r] % MOD * fi[n-r] % MOD;
}
```

Intuition. $C(n, r)$ counts r -element choices: there are $n \cdot (n-1) \cdots (n-r+1)$ ordered picks, and each unordered choice was counted $r!$ times — divide the overcount away; with factorials that's $n! / (r!(n-r)!)$. The downward walk for inverse factorials is one identity used n times: $1/(i-1)! = i \cdot (1/i!)$, so one expensive modular inverse (of $f[N-1]$) seeds all the others in $O(N)$. Pascal's rule $C(n, r) = C(n-1, r-1) + C(n-1, r)$ — "the n -th item is in or out" — is the fallback table when $n \leq \sim 5000$ or the modulus isn't prime (no inverses needed, just additions).

- Make C return 0 out of range instead of asserting — identities like Vandermonde sums run r over ranges where some terms are legitimately zero.

2.23 Stars and bars

Triggers. "Number of ways to write n as an **ordered** sum of k non-negative integers"; "distribute n identical candies among k children"; "count multisets of size n from k types". The phrase "identical items, distinct boxes" is the tell.

```
// solutions of  $x_1 + x_2 + \dots + x_k = n$ ,  $x_i \geq 0$ :  $C(n + k - 1, k - 1)$ 
// solutions with every  $x_i \geq 1$ :  $C(n - 1, k - 1)$ 
// multisets of size  $n$  drawn from  $k$  types:  $C(n + k - 1, n)$  (same number)
ll ways = C(n + k - 1, k - 1);
ll ways_pos = C(n - 1, k - 1);
```

Intuition. Encode a distribution as a picture: n stars in a row, and $k-1$ bars chopping the row into k segments — segment i 's star count is x_i . Example, $n = 5, k = 3$: $\star\star|\star|\star\star$ means $(2, 1, 2)$; $||\star\star\star\star\star$ means $(0, 0, 5)$. Every distribution draws exactly one picture and every picture decodes to exactly one distribution — a bijection, so count pictures instead: among $n + k - 1$ symbol slots, choose which $k - 1$ hold bars. For the "every $x_i \geq 1$ " variant, hand each variable one star up front (substitute $y_i = x_i - 1$) and reuse the formula on $n - k$ stars. Upper bounds ($x_i \leq c$) don't have a one-shot formula — that's stars and bars composed with inclusion-exclusion (§2.26): subtract distributions where some variable got overfed $c + 1$ in advance.

- Recognition practice: "non-decreasing sequences of length k with values in $[1, m]$ " = multisets = $C(m + k - 1, k)$. Many disguises reduce to choosing a multiset.

2.24 Catalan numbers

Triggers. "Balanced bracket sequences", "every prefix has at least as many openings as closings", "paths that never dip below zero / cross the diagonal", "non-crossing" pairings or chords, counting binary trees, triangulations. Sequence starts 1, 1, 2, 5, 14, 42, 132, 429, ...

```
//  $Cat(n) = C(2n, n) / (n + 1) = C(2n, n) - C(2n, n + 1) = (2n)! / (n! (n+1)!)$ 
ll cat(ll n) { return f[2*n] * fi[n] % MOD * fi[n+1] % MOD; }
```

Intuition. One mental anchor: **n correctly nested bracket pairs**; everything else is that object in costume. An up/down path that never goes negative is a bracket sequence read as ± 1 . A binary tree is a bracket sequence via "(left-subtree) right-subtree". The recurrence comes from the **first-return decomposition**: any balanced sequence is uniquely "(A)B" where A is what sits inside the first bracket's matching partner and B is what follows — both independently balanced — giving $Cat(n+1) = \sum Cat(i) \cdot Cat(n-i)$. The closed form $Cat(n) = C(2n, n) - C(2n, n+1)$ reads as good = all – bad: among all $C(2n, n)$ up/down paths, the bad ones (those that dip below zero) turn out to correspond one-for-one with arbitrary paths of a slightly different step count, $C(2n, n+1)$. The machine behind that correspondence — reflecting the path at its first sin — is the reflection principle, treated properly at Tier 3; at this tier, command the objects, the recurrence, and the formula.

- Spot-check tool: if a counting answer for small cases runs 1, 2, 5, 14, 42 — stop deriving, it's Catalan. Brute force small n (§0.3) and compare against the sequence; this identifies the structure in minutes.

2.25 Derangements

Triggers. "Permutations where nothing stays in its place" — hats returned so nobody gets their own, letters in wrong envelopes, secret-santa with no self-gifting. Written ! n : 1, 0, 1, 2, 9, 44, 265, ...

```
vector<ll> D(n + 1);
D[0] = 1; if (n >= 1) D[1] = 0;
for (int i = 2; i <= n; i++)
    D[i] = (i - 1) * ((D[i-1] + D[i-2]) % MOD) % MOD;
// closed form:  $!n = n! \cdot \sum_{k=0..n} (-1)^k / k! \sim n! / e$  (about 36.8% of all perms)
```

Intuition. The recurrence is a two-case story. Element 1 must land somewhere else — say position j , with $(n-1)$ choices. Now where does j go? Case A: j goes to 1 — the two of them swapped; the remaining $n-2$ elements just need their own derangement: $!(n-2)$. Case B: j does *not* go to 1 — then j is an element forbidden from exactly one position (position 1), which is structurally identical to an

ordinary element in a derangement of the remaining $n-1$: $!(n-1)$. Hence $!n = (n-1)(!(n-1) + !(n-2))$. The closed form is inclusion-exclusion (§2.26) verbatim: from all $n!$ permutations subtract those fixing at least one chosen point — $C(n,k) \cdot (n-k)!$ permutations fix a given set of k points, and the alternating sum telescopes into $n! \cdot \sum (-1)^k / k!$. That sum is the series for e^{-1} , hence the lovely fact that a random permutation is fixed-point-free with probability $\rightarrow 1/e$.

2.26 Inclusion-exclusion

Triggers. "Count objects with **none** of these bad properties" / "at least one"; constraints that are easy to *force* but hard to *forbid* (divisibility, containing a pattern, exceeding a cap). The complement move: $\text{valid} = \text{total} - |\text{bad}_1 \cup \text{bad}_2 \cup \dots|$.

```
// |A1 ∪ A2 ∪ A3| = S1 - S2 + S3 (sizes of single / pairwise / triple intersections)
// general: count with NO property = sum over subsets T of properties of (-1)^|T| * N(force all of T)

// example: how many x in [1, N] are divisible by NONE of the primes P[0..k)
ll ans = 0;
for (int m = 0; m < (1 << k); m++) {
    ll d = 1; bool skip = false;
    for (int i = 0; i < k; i++) if (m >> i & 1) {
        d *= P[i];
        if (d > N) { skip = true; break; } // term contributes 0; also avoids overflow
    }
    if (skip) continue;
    ans += (popcount((unsigned)m) & 1 ? -1LL : 1LL) * (N / d);
}
```

Intuition. Why the alternating signs are exactly right: audit one object that has t bad properties. Across all 2^k subset terms it is counted only by subsets of the properties it actually has, with sign $(-1)^{|T|}$ — total contribution $\sum_j (-1)^j C(t, j) = (1 - 1)^t$, which is 1 if $t = 0$ and 0 if $t \geq 1$. So the alternating sum is a filter that keeps clean objects with weight exactly 1 and annihilates everything else; the layered "add singles, oops, subtract pairs, oops, add triples" Venn story is this identity unrolled. Practical consequence of the formula shape: you only need to count objects with **at least** a forced set of properties, $N(\supseteq T)$ — usually the easy direction (divisible by $d = \text{product}$; cap broken by handing out $c+1$ in advance; letter placed in its own envelope).

- Workhorse combos at this level: derangements (§2.25); stars and bars with upper bounds (§2.23); surjections ("every box non-empty"): $\sum_j (-1)^j C(k, j) (k-j)^n$ — forbid j chosen boxes, count freely.
- Cost is $O(2^k \cdot \text{work})$ over the *properties*, not the objects — IE wants few properties ($k \leq \sim 20$), arbitrarily many objects.

2.27 Overflow awareness — long long, size_t, __int128

Triggers for a pre-submit audit, every single problem: any multiplication, any sum over 10^5+ terms, any $1 < k$, any `.size()` arithmetic. Limits to know cold: $\text{int} \approx 2.1 \cdot 10^9$, $\text{long long} \approx 9.2 \cdot 10^{18}$.

```
int a = 1e9, b = 1e9;
ll bad = a * b; // OVERFLOW: multiplied as int, THEN widened. UB.
ll good = (ll)a * b; // widen first

ll big = 1LL << 40; // 1 << 40 would be int UB (§1.3)

for (int i = 0; i + 1 < (int)v.size(); i++) ... // v.size()-1 wraps to 2^64-1 when empty!
for (int i = 0; i + 1 < ssize(v); i++) ... // C++20: signed size, no trap

// products of two longs (~1e18 * 1e18): compare via __int128
if ((__int128)a * b > (__int128)c * d) ... // compute & compare; printing needs a helper
```

- Signed overflow is undefined behavior, not wraparound — `-fsanitize=undefined` (§0.1) catches it locally; on the judge it's a silent wrong answer.

- Estimate the maximum value of every quantity at design time: $n = 2 \cdot 10^5$ values up to $10^9 \rightarrow$ sums reach $2 \cdot 10^{14}$ (ll fine), sums of pairwise products reach $\sim 10^{28}$ (nothing native fits $\rightarrow$ the problem must intend a mod, or you must restructure).
- Habit that prevents 90% of this: any arithmetic that *could* exceed int range is done in ll variables end-to-end. Mixed int/ll expressions are where the bugs breed.

2.28 Divisors: counting, summing, and harmonic enumeration

Triggers. "Number/sum of divisors", "for every pair (i, j) with i dividing j", per-divisor aggregation over all numbers up to n.

```
// all divisors of one n in O(sqrt n) -- in pairs (d, n/d)
vector<ll> divs;
for (ll d = 1; d * d <= n; d++)
    if (n % d == 0) {
        divs.push_back(d);
        if (d != n / d) divs.push_back(n / d);
    }

// from the factorization n = p1^e1 * p2^e2 * ...
// count: d(n) = (e1+1)(e2+1)...      sum: sigma(n) = prod (p^(e+1) - 1) / (p - 1)

// harmonic enumeration: visit every (divisor d, multiple m) pair for all m <= N
for (int d = 1; d <= N; d++)
    for (int m = d; m <= N; m += d) {
        // d divides m -- e.g. cnt_divisors[m]++
    }
// total work = N/1 + N/2 + ... + N/N = O(N log N)
```

Intuition. A divisor is a choice of exponent $0 \dots e_i$ for each prime, independently — the product rule gives $d(n)$ immediately, and $\sigma(n)$ factors into a product of geometric series $(1 + p + \dots + p^e)$ for the same reason: expanding the product generates every divisor exactly once. The harmonic double loop looks quadratic but the inner loop runs N/d times; summing N/d over d is N times the harmonic series $\approx N \ln N$. This is the cheap way to compute *something about all divisors of all numbers up to N at once* — divisor counts, smallest prime tables, "for every g, how many pairs have gcd divisible by g" — and it is the precursor of half the number-theoretic sieving tricks at higher tiers. Internalize the sum $\sum N/d = O(N \log N)$; you will budget with it constantly.

2.29 Strings: brute-force matching and palindromes

Triggers. "Does pattern p occur in text t / where", "is it (almost) a palindrome", "count palindromic substrings". At this tier brute force and center-expansion are the honest tools; hashing (§2.30) upgrades them, KMP/Z (Tier 3) make matching linear.

```
// brute-force matching: O(n*m) -- fine while n*m <= ~10^7-10^8
vector<int> occ;
for (int i = 0; i + m <= n; i++) {
    bool ok = true;
    for (int j = 0; j < m && ok; j++) ok = (t[i + j] == p[j]);
    if (ok) occ.push_back(i);
}

// palindrome check: two pointers
bool pal = true;
for (int i = 0, j = n - 1; i < j; i++, j--) if (s[i] != s[j]) pal = false;

// count ALL palindromic substrings in O(n^2): expand around each center
ll cnt = 0;
for (int c = 0; c < 2 * n - 1; c++) {
    // n odd centers + n-1 even centers
    int l = c / 2, r = l + c % 2;
    while (l >= 0 && r < n && s[l] == s[r]) cnt++, l--, r++;
}
```

Intuition. The center trick: every palindrome is symmetric around a center — a character (odd length) or a gap between characters (even length), $2n-1$ centers total. Growing outward from a center, palindromicity is monotone: the moment a mismatch appears, no longer palindrome at this center exists. So each center is an independent two-pointer expansion. "Almost-palindrome with one deletion allowed": run the two-pointer check; at the first mismatch, branch into skipping either side once — a 1500-rated classic.

2.30 Polynomial hashing

Triggers. Compare substrings in $O(1)$, many times: "are these two ranges equal", count positions where a pattern matches, check palindromes per query (hash vs reversed hash), group equal strings. The all-purpose string tool before suffix structures exist in your toolkit.

```
struct Hash {
    static const ll M = 1'000'000'007;    // a large prime modulus
    ll B;                                // RANDOM base -- see pitfalls
    vector<ll> h, pw;                     // h[i] = hash of prefix s[0..i)
    Hash(const string& s) {
        mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
        B = uniform_int_distribution<ll>(300, M - 2)(rng);
        int n = s.size();
        h.assign(n + 1, 0); pw.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = (h[i] * B + s[i]) % M;
            pw[i + 1] = pw[i] * B % M;
        }
    }
    ll get(int l, int r) const {          // hash of s[l..r), 0(1)
        return ((h[r] - h[l] * pw[r - l]) % M + M) % M;
    }
};
// usage: Hash ht(t), hp(p); ht.get(i, i + m) == hp.get(0, m) -> match at i (probably)
```

Intuition. Read the string as a number written in base B , digit by digit: h grows by "shift left, add next character" — exactly how you'd read 4,712 from its digits. A substring's value is then extracted like middle digits of a number: take the prefix ending at r , and subtract the prefix ending at l *shifted left* by the substring's length — $h[r] - h[l] \cdot B^{(r-l)}$. All of it mod M because the true numbers are astronomically long. Equal strings always hash equal; the price is that unequal strings collide with probability $\approx 1/M$ per comparison — fine for $\sim 10^5$ – 10^6 comparisons under $M \approx 10^9$, and fixable with a second independent (B, M) pair when comparisons multiply (double hashing, Tier 3).

- **Randomize B.** On Codeforces, fixed published bases get hacked with crafted anti-tests. A random base per run makes the adversary blind. Never hash with "mod 2^{64} via unsigned overflow" and a fixed base — a known construction (Thue-Morse strings) breaks it deterministically.
- Compare hashes only from the *same* Hash object (same B, M). For two strings, build both with the same generated B — e.g., make B a global initialized once.
- Equality of hashes is evidence, not proof. In a final answer where a false positive is fatal and cheap to avoid, verify the candidate match directly once.

2.31 Coordinate compression

Triggers. Values range to $10^9/10^{18}$ but only the $n \leq 2 \cdot 10^5$ *given* values matter, and you want them as array indices (frequency tables, later Fenwick/segment trees). Order matters; magnitudes don't.

```
vector<ll> xs = a;                                // collect EVERY value you will ever query
ranges::sort(xs);
xs.erase(unique(xs.begin(), xs.end()), xs.end());
auto id = [&](ll x) {                             // value -> rank in [0, xs.size())
    return int(lower_bound(xs.begin(), xs.end(), x) - xs.begin());
};
// a[i] -> id(a[i]); xs[r] recovers the original value from rank r
```

Intuition. If the algorithm only ever asks "<, =, >" questions about values, the values are just labels — replace each by its rank and nothing changes except that the universe shrinks from 10^{18} to n . The one discipline: compress the union of *all* values that will ever be touched (array values **and** query endpoints), or a later `lower_bound` maps an unseen value onto someone else's rank.

2.32 Indexing and floating-point conventions

- **Half-open ranges** `[l, r)` **everywhere**. Lengths are $r - l$, empty is $l == r$, splitting at m gives `[l, m) + [m, r)` with no ± 1 — the same conventions the STL itself uses. Convert inclusive statements at the parsing boundary and think half-open internally.
- **0-indexed by default; 1-indexed with a zero pad** when a sentinel simplifies formulas (prefix sums §1.8/§2.18). Pick per problem, write it in a comment, never mix.
- **Avoid floats when integers can do:** compare fractions by cross-multiplying in `ll` ($a*d < c*b$, mind signs), areas via cross products (Tier 3), distances by comparing *squared* values.

```
cout << fixed << setprecision(10);           // required output precision: set once
const double EPS = 1e-9;
bool eq(double a, double b) { return fabs(a - b) < EPS; } // never == on doubles
// doubles hold ~15-16 significant digits; ll values beyond 2^53 lose exactness in double
```

2.33 Custom structs and comparators

Triggers. Entities with several fields you sort, store in sets, or push through priority queues. Modern C++ removes nearly all the boilerplate.

```
struct Item {
    int w, v;
    // C++20: one line generates ==, <, >, <=, >= -- lexicographic by (w, then v)
    auto operator<=>(const Item&) const = default;
};

ranges::sort(items); // uses <=> above
ranges::sort(items, {}, &Item::v); // by a single field (projection)
ranges::sort(items, [](const Item& a, const Item& b) {
    return a.w != b.w ? a.w < b.w : a.v > b.v; // custom: w asc, tie v desc
});

priority_queue<Item, vector<Item>, greater<>> pq; // min-heap via the generated >

// stateless lambda as a set comparator (C++20: default-constructible)
auto byV = [](const Item& a, const Item& b) { return a.v < b.v; };
set<Item, decltype(byV)> s;
```

- The strict-weak-ordering contract from §1.4 applies to every comparator here too: no `<=`, equal elements must compare false both ways. In a set, elements "equal under the comparator" are *the same element* — a set ordered only by `.v` silently rejects a second item with the same `v`. Include a tiebreaker field (like an index) if duplicates must coexist.

2.34 Interactive problems

Triggers. The statement says "This is an interactive problem": you print queries, a judge answers, under a query budget. Almost always the budget is $\sim \log_2(\text{range})$ — i.e., the intended solution is a binary search *conversation* (§1.5 with the predicate answered live).

```
// guess a hidden x in [1, n]; query "? m" answers "<" or ">="; budget ~log2 n
int lo = 1, hi = n; // invariant: x in [lo, hi]
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    cout << "? " << mid << endl; // endl = print AND FLUSH (required here)
    string ans; cin >> ans; // read the judge's reply
    if (ans == "<") hi = mid - 1; // adapt to the protocol of the problem
    else lo = mid + 1; // (this split matches "< mid" / ">= mid")
}
cout << "! " << lo << endl;
```

- **Flush after every query** — `endl` or `cout.flush()`. Without it your output sits in a buffer, the judge waits forever, and you get "Idleness limit exceeded". This is the one context where §0.1's "never endl" rule inverts.
- Count your queries against the budget before coding: a budget of 20 over $n = 10^6$ says binary search ($2^{20} \approx 10^6$); a budget of $\sim 2n$ says a linear scan with one trick. The budget is the strongest hint in the statement.
- **Adaptive judges** may not fix x in advance — they answer consistently but adversarially. Consequence: your *worst case* must fit the budget, not the average; randomized "probably fast" strategies can be forced into their bad case.
- If the judge ever replies -1 (invalid query / limit exceeded), exit immediately — keep reading and you trade a clean verdict for a confusing TLE.

Exit checklist — when Tiers 1 and 2 are actually done

From the roadmap's "Done when" gates; be honest, this is where months are saved or lost.

- **Tier 1:** every §1.x item codes from memory in under 3 minutes. CF 1200–1400 problems fall in under 15 minutes. Binary search, two pointers, prefix sums are reflexes — you reach for them before consciously deciding. You rarely TLE/RE on Tier-1 content.
- **Tier 2:** every §2.x item codes from memory in under 5 minutes with edge cases handled. You finish Div. 3 contests. CF 1600–1800 problems solve in under ~ 30 minutes most of the time.
- **Practice map:** Tier 1 → CSES Introductory + Sorting and Searching, CF 1000–1400. Tier 2 → CSES Graph Algorithms (basic) + Dynamic Programming, AtCoder DP Contest (A–J), CF 1400–1800 by tag, one virtual Div. 3 per week. Keep the error log; review it before every contest.

Next: Tier 3 — segment trees, Dijkstra, tree/bitmask/digit DP, KMP, and parametric search. Everything there stands on the intuitions built here: amortized monotone structures, DAG-ordered DP, and prefix-style precomputation.